



FEBRUARY 10, 2026

MINION ME TEST PLAN

SANDRA WAHU NGUGI

18/03030

BISF 4500 FINAL YEAR PROJECT 2



Table of Contents

1.0	Introduction.....	3
1.1	Goals and Objectives.....	3
1.2	Statement of Scope.....	3
	Features and Functionalities to be tested.....	3
	Features and Functionalities NOT to be tested.....	4
1.3	Major Constraints.....	4
2.0	Test Plan.....	4
2.1	Software (SCI's) to be tested	4
2.2	Testing Strategy.....	5
2.2.1	Unit testing	5
2.2.2	Integration Testing.....	5
2.2.3	Validation Testing	6
2.2.4	High-Level Testing.....	6
2.3	Testing Tools and Environment	6
2.4	Test Records.....	7
2.5	Test Metrics.....	7
2.6	Testing Schedule	8
3.0	Detailed Test Procedure.....	8
3.1	Software (SCI's) to be tested	8
3.2	Testing Procedure.....	8
3.2.1	Unit Test Cases.....	9
	Component 1: Authentication Module.....	9
3.2.1.2	Stubs and Drivers for Authentication Module.....	9
3.2.1.3	Test Cases – Authentication Module.....	9
3.2.1.4	Purpose of Tests – Authentication Module	10
3.2.1.5	Expected Results – Authentication Module	10
	Component 2: Task Management Module.....	10
3.2.1.2	Stubs and/or Drivers for Task Module	10
3.2.1.3	Test Cases – Task Module.....	10

3.2.1.4 Purpose of Tests – Task Module	11
3.2.1.5 Expected Results – Task Module	11
3.2.2 Integration Testing.....	11
3.2.2.1 Testing Procedure for Integration.....	11
3.2.2.2 Stubs and Drivers Required.....	11
3.2.2.3 Test Cases and Their Purpose.....	12
3.2.2.4 Expected Results.....	12
3.2.3 Validation Testing	12
3.2.3.1 Testing Procedure for Validation	13
3.2.3.3 Expected Results.....	13
3.2.3.4 Pass/Fail Criterion for All Validation Tests	13
3.2.4 High-Order Testing.....	13
3.2.4.1 Recovery Testing.....	14
3.2.4.2 Security Testing.....	14
3.2.4.3 Stress Testing.....	15
3.2.4.4 Performance Testing.....	15
3.2.4.5 Alpha/Beta Testing.....	16
3.2.4.6 Pass/Fail Criterion for All System Tests	16
3.3 Testing Resources and Staffing.....	17
3.4 Test Work Products.....	17
3.5 Test Record Keeping and Test Log.....	17

1.0 Introduction

This document outlines the testing plan and procedures for the Minion Me service marketplace, which serves as a connection between clients and assistants (“Minions”) for various everyday tasks. As the only developer involved in this project, I handle the complete development lifecycle, including implementation, testing, debugging, and final validation prior to submission. The goal of testing is to confirm that the system not only functions correctly but also operates reliably during typical use. In addition to functionality, testing guarantees that the platform maintains stability and meets the requirements established for this final year project.

1.1 Goals and Objectives.

1. To verify that Minion Me fulfills the functional criteria of a service marketplace.
2. To identify and record bugs and defects promptly, and then confirm fixes through testing.
3. To make sure the system is intuitive, secure, and reliable for presentation.
4. To confirm that the main user journeys, specifically for Clients and Minions, operate smoothly.

1.2 Statement of Scope

Features and Functionalities to be tested

- ✓ User registration and login for Client and ‘Minion’ accounts.
- ✓ Authentication and session handling.
- ✓ User profiles for Clients and Minions.
- ✓ Marketplace browsing for tasks.
- ✓ Task lifecycle:
Create task → view task → accept/assign → progress → complete/cancel
- ✓ Communication in the platform: notifications of tasks submitted by clients and tasks accepted by ‘Minions’.
- ✓ Ratings and reviews after a task is completed.

- ✓ Admin functions.
- ✓ Database persistence and API correctness.

Features and Functionalities NOT to be tested

- ✓ Internal behavior of third-party systems, i.e., real M-Pesa Services.
- ✓ Live production performance across many regions and devices.
- ✓ Penetration testing beyond academic scope. Only basic website security checks are included.
- ✓ Features not implemented in the project.

1.3 Major Constraints

- ✓ Limited time and staffing.
- ✓ Limited access to real payment gateways.
- ✓ Single test environment that may differ from “real-world” deployment.
- ✓ Limited device coverage.

2.0 Test Plan

This document serves as a guide for the comprehensive testing strategy and the project management challenges necessary for carrying out efficient tests.

2.1 Software (SCI's) to be tested

System Name: Minion Me Service Marketplace.

Components included:

- Frontend UI.
- Backend services and APIs.
- Database.
- Authentication subsystem.
- Admin interface.

Exclusions: Third-party service providers, such as email services, SMS, and M-Pesa payments.

2.2 Testing Strategy

Testing will be carried out in a step-by-step manner. Individual components will first be verified in isolation to ensure they function correctly. Once confirmed, these components will be integrated and tested together to validate communication between modules. Finally, the entire web application will be evaluated as a complete system to ensure overall stability and consistency.

2.2.1 Unit testing

I will test the following parts of the system individually.

- The login function
- The task creation logic
- Form validation for sign-up, login, and task creation forms.
- All database queries

This helps me catch bugs early.

2.2.2 Integration Testing

After testing individual parts, I will test how they work together.

Examples:

- User signs up → logs in → posts a task
- Minion logs in → views task → accepts task

- Task status updates correctly in the database

This ensures different parts of the system communicate properly.

2.2.3 Validation Testing

I will test the system based on the project requirements.

I will go through real-life usage scenarios systematically to confirm:

- ✓ A client can successfully post a task
- ✓ A minion can accept and complete a task
- ✓ The system stores all actions correctly

2.2.4 High-Level Testing

I will also test the system as a whole.

This includes:

- Checking that the system does not crash
- Testing basic security, for example, making sure users cannot edit other users' tasks
- Testing performance by checking that pages load in a reasonable time
- Restarting the server to confirm data is not lost

I may also ask a few peers to try the system and give feedback.

2.3 Testing Tools and Environment

Testing will be done using:

- Local development environment

- Browser Developer Tools
- Postman for API testing
- PgAdmin 4 database viewer
- Manual testing
- Burp Suite for penetration testing

2.4 Test Records

I will keep a simple test log that includes:

- Date
- Feature tested
- Expected result
- Actual result
- Status (Pass/Fail)
- Notes or screenshots if needed

All errors found will be recorded and fixed before the final presentation.

2.5 Test Metrics

To measure testing progress, I will track:

- Number of features tested
- Number of errors found
- Number of errors fixed
- Final pass rate of all core features

The system will be considered ready when all major features pass testing.

2.6 Testing Schedule

Testing will happen alongside development, following the given criteria below.

- During development, Unit testing
- After major features are built, Integration testing
- Before submission, Full system testing and final validation

3.0 Detailed Test Procedure

This section explains how testing will actually be performed in detail.

3.1 Software (SCI's) to be tested

The software to be tested is:

- Minion Me Web Application (Frontend)
- Minion Me Backend APIs
- Database side of Minion Me that uses PostgreSQL
- Authentication Module

Exclusions

- Internal systems of third-party services (e.g., M-Pesa integration)
- Features not implemented in this project

3.2 Testing Procedure

I will perform testing in stages. Each stage focuses on a different level of the system, starting from individual components and moving to the complete system.

3.2.1 Unit Test Cases

I intend to perform unit tests on individual software components listed below. Each major component is to be tested independently before integration.

The components selected for unit testing are:

- Authentication Module
- User Profile Module
- Task Management Module

Each component is further described below.

Component 1: Authentication Module

3.2.1.2 Stubs and Drivers for Authentication Module

- Mock database connection for user data
- Mock token generation
- Simulated HTTP requests for login and registration

3.2.1.3 Test Cases – Authentication Module

1. Register a user with valid details
2. Register a user with an existing email
3. Log in with correct credentials
4. Log in with an incorrect password
5. Access a protected route without authentication

3.2.1.4 Purpose of Tests – Authentication Module

This is done to confirm that user registration and login work correctly, and that unauthorized access is prevented.

3.2.1.5 Expected Results – Authentication Module

- Valid users are successfully registered and logged in
- Duplicate emails are rejected
- Incorrect passwords are rejected
- Protected routes require authentication to access

Component 2: Task Management Module

3.2.1.2 Stubs and/or Drivers for Task Module

- Mock authenticated user
- Mock database operations for task storage

3.2.1.3 Test Cases – Task Module

- Create task with valid input
- Create task with missing required fields
- Update task status
- Delete task
- Unauthorized user attempts to edit a task

3.2.1.4 Purpose of Tests – Task Module

This is done to ensure task creation, updating, deletion, and authorization logic work correctly.

3.2.1.5 Expected Results – Task Module

- Tasks are stored correctly in the database
- Invalid inputs are rejected
- Only authorized users can modify tasks
- Task status changes are saved correctly

3.2.2 Integration Testing

Integration testing verifies that different modules work together correctly and systematically.

3.2.2.1 Testing Procedure for Integration

1. Deploy backend and database together
2. Connect frontend to backend
3. Perform complete user workflows
4. Verify database records after each workflow

3.2.2.2 Stubs and Drivers Required

Since third-party systems are excluded from testing, integration testing will focus only on internal system components.

If any external services are simulated, such as payment confirmation or notifications, they will be represented using simple mock responses within the system.

No direct testing of external service provider infrastructure will be conducted.

3.2.2.3 Test Cases and Their Purpose

- ✓ Client signs up → logs in → posts task

Purpose: Verify full client workflow

- ✓ Minion signs up → views tasks → accepts task

Purpose: Verify full minion workflow

- ✓ Client marks task as completed

Purpose: Verify task lifecycle completion

3.2.2.4 Expected Results

- All modules communicate correctly
- Task status updates correctly
- No data inconsistency in database
- Role-based permissions are enforced

3.2.3 Validation Testing

This section ensures the system, Minion Me, meets the project requirements.

3.2.3.1 Testing Procedure for Validation

- Review project requirements
- Execute end-to-end scenarios
- Compare actual system behavior with expected requirements
- Record results in test log

3.2.3.3 Expected Results

- All major project requirements are satisfied
- Core user journeys and workflow are error-free

3.2.3.4 Pass/Fail Criterion for All Validation Tests

Pass:

All core features, such as registration, login, posting tasks, accepting tasks, and completing tasks, function correctly.

Fail:

Any critical feature does not function correctly.

3.2.4 High-Order Testing

System testing evaluates the system as a whole.

3.2.4.1 Recovery Testing

Procedure:

- Restart the server during operation
- Verify that data is not lost

Purpose:

- Ensure the system can recover from interruptions

Pass Criteria:

- No data corruption
- The system restarts successfully

3.2.4.2 Security Testing

Procedure:

- Attempt simple SQL injection
- Attempt unauthorized access to other users' tasks
- Verify passwords are hashed
- Run the system through Burp Suite

Purpose:

- Ensure basic application security
- Evaluate risk score

Pass Criteria:

- Unauthorized actions are blocked

- Passwords are not stored in plain text
- Low security risk score

3.2.4.3 Stress Testing

Procedure:

- Create multiple tasks
- Perform repeated login attempts

Purpose:

- Observe system behavior under moderate load

Pass Criteria:

- System remains responsive
- No crashes occur

3.2.4.4 Performance Testing

Procedure:

- Measure response time for login
- Measure response time for task listing

Purpose:

- Ensure acceptable performance

Pass Criteria:

- System responds within reasonable time

3.2.4.5 Alpha/Beta Testing

Procedure:

- Allow selected peers to test the system
- Collect feedback

Purpose:

- Identify usability issues
- Get feedback from a simulation of real users

Pass Criteria:

- No critical usability problems

3.2.4.6 Pass/Fail Criterion for All System Tests

Pass:

- No critical bugs remain
- System operates correctly under normal use

Fail:

- Any issue that prevents users from completing core tasks

3.3 Testing Resources and Staffing

- ✓ Developer: Myself
- ✓ Tester: Myself
- ✓ Beta Testers: Selected peers
- ✓ Supervisor: ***Clive Onsomu***

No external ITG team is involved.

3.4 Test Work Products

The following documents will be produced:

- Test Plan
- Test Cases
- Test Log
- Defect Log
- Final Test Summary Report

3.5 Test Record Keeping and Test Log

A test log will be maintained, including:

- Date of test
- Component tested
- Test case ID
- Expected result
- Actual result
- Status (Pass/Fail)

- Notes or screenshots

All test results will be stored for reference and submission if required.